

# Cost & Latency Optimization Playbook

Production GenAI, RAG, Agents, APIs, LLM Infrastructure, and Enterprise AI Deployment  
Edition

By Tobias Weissmann

*A senior-level field engineering guide for GenAI FDE interviews, customer deployments, architecture reviews,  
and enterprise optimization decisions*

## Designed for

GenAI Forward Deployed Engineers | Applied AI Engineers | AI Solutions Engineers | GenAI Consultants | AI  
Platform Engineers | AI Solutions Architects | Backend and Platform Engineers building LLM systems

## How to use this playbook

Use it as an interview preparation manual, a customer discovery checklist, a production incident guide, and a vocabulary bank for senior customer-facing AI engineering conversations. The strongest candidates do not say, "Use a faster model." They decompose cost, latency, quality, risk, and business value into measurable components.

## Detailed Table of Contents

---

1. Why Cost and Latency Matter in GenAI FDE Work
2. Core Cost Drivers in GenAI Systems
3. Core Latency Drivers in GenAI Systems
4. Cost vs Latency vs Quality Trade-Off Framework
5. Model Selection Optimization
6. Prompt Optimization for Cost and Latency
7. RAG Cost and Latency Optimization
8. Agent Cost and Latency Optimization
9. Caching Strategies
10. Streaming and User Experience Optimization
11. Infrastructure and Deployment Optimization
12. Observability for Cost and Latency
13. Cost Budgeting and Guardrails
14. Enterprise Customer Communication
15. Production Troubleshooting Scenarios
16. Interview Case Studies
17. Checklists
18. Final Senior-Level Mental Model

### Core optimization equation

End-to-end latency = auth + request routing + retrieval + reranking + prompt construction + model queue time + inference + tool/API calls + post-processing + network return.

Cost per request = input-token cost + output-token cost + embedding/retrieval cost + reranking cost + tool/API cost + infra allocation + logging/evaluation overhead.

Senior FDE principle: never optimize the model in isolation. Break the workflow into measured components, then remove the largest bottleneck first.

# 1. Why Cost and Latency Matter in GenAI FDE Work

In customer-facing GenAI work, cost and latency are not secondary engineering concerns. They determine whether users adopt the system, whether executives approve expansion, whether the pilot survives procurement review, and whether the deployment can scale beyond a controlled demo. A GenAI FDE must be able to speak simultaneously to engineers, product owners, security teams, procurement, executives, and end users.

## Why demo systems often fail at production scale

- **Demos hide traffic shape:** A demo usually has one user, one question, warm caches, a clean dataset, no permission complexity, and no adverse load. Production has many tenants, mixed request types, document churn, auth checks, retries, and concurrent users.
- **Demos hide tail latency:** Average latency may look acceptable, but P95/P99 latency determines real user frustration. A demo path that takes 3 seconds may become 20 seconds when retrieval, reranking, tool calls, and provider queueing are included.
- **Demos hide usage expansion:** Users ask longer questions, upload larger documents, and use the system in ways the team did not anticipate. Token usage and tool calls grow faster than request count.
- **Demos hide governance cost:** Security reviews, audit logging, evaluation, human review, and support workflows add cost that prototype teams often ignore.

## Why LLM cost can explode after rollout

LLM cost grows with token volume, output length, model tier, retries, agent loops, evaluation jobs, embeddings, vector search, reranking, logging, and provider/infrastructure overhead. A common mistake is to multiply expected daily users by one model call. Real systems often call multiple models per user request: routing, query rewriting, retrieval, reranking, answer generation, evaluation, summarization, safety checks, and sometimes tool execution.

## Why latency kills user adoption

Users tolerate latency when the task is complex, infrequent, and high value. They reject latency when the task is conversational, repetitive, operational, or embedded into a fast workflow. In enterprise systems, latency also creates political risk: if the first pilot feels slow, stakeholders may conclude that GenAI is not ready, even when answer quality is strong.

## Executive optimization lens

| Executive concern | What they really ask                             | FDE translation                                                                                                     |
|-------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| ROI               | Does the value justify the spend?                | Track cost per resolved ticket, cost per document processed, hours saved, deflection rate, and revenue/risk impact. |
| Predictability    | Can we budget this safely?                       | Add per-tenant budgets, token limits, quotas, anomaly alerts, and model-routing rules.                              |
| Adoption          | Will employees actually use it?                  | Optimize perceived latency, workflow fit, reliability, and explainability.                                          |
| Risk              | Will this leak data or create compliance issues? | Use permission-aware retrieval, audit logs, caching controls, and human approval for sensitive actions.             |
| Scalability       | Can this work beyond the pilot?                  | Load-test realistic traffic, measure P95/P99, and define cost curves before rollout.                                |

## Prototype latency vs production latency

| Dimension     | Prototype                              | Production                                                                     |
|---------------|----------------------------------------|--------------------------------------------------------------------------------|
| Dataset       | Small, manually curated, often static. | Large, messy, multi-tenant, constantly changing.                               |
| Security      | Minimal or mocked.                     | OAuth/OIDC/SAML, RBAC/ABAC, permission filtering, audit logging.               |
| Load          | Single-user manual test.               | Concurrent users, traffic spikes, batch jobs, retries.                         |
| Observability | Console logs.                          | Traces, token metrics, cost dashboards, incident alerts.                       |
| Latency       | One happy path.                        | Tail latency, provider queueing, cold starts, tool/API delays.                 |
| Cost          | Small usage bill.                      | Budget ownership, cost allocation, anomaly detection, procurement constraints. |

### Interview-ready explanation

A strong GenAI FDE might say: "I would not optimize the model first. I would break latency into retrieval time, reranker time, prompt construction, first-token latency, generation time, tool-call time, and post-processing. For cost, I would separate input tokens, output tokens, retries, embeddings, reranking, tool calls, logging, evaluation, and infrastructure. Then I would optimize the largest measured driver while protecting answer quality and security."

## 2. Core Cost Drivers in GenAI Systems

Cost in GenAI systems is usually a portfolio of small costs that become expensive at scale. The dangerous drivers are the hidden multipliers: retries, long prompts, long outputs, agent loops, repeated retrieval, large context windows, and evaluation pipelines running silently in the background.

| Cost driver              | What causes it                                                     | Why underestimated                                                                 | Production signal                                                | How to reduce                                                                       | Trade-off                                                      |
|--------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Input tokens             | Long prompts, retrieved context, chat history, tool results.       | Teams count user question but ignore system prompt, RAG chunks and hidden history. | Bill increases even when user traffic is flat.                   | Trim prompts, compress context, summarize history, cache static prompt parts.       | Less context can reduce accuracy if retrieval quality is weak. |
| Output tokens            | Verbose answers, uncontrolled generation, reasoning-heavy tasks.   | Teams measure requests, not generated tokens.                                      | Long answers dominate per-request cost.                          | Set max tokens, concise templates, structured output, summarize only when needed.   | Too aggressive limits can produce incomplete answers.          |
| Long context windows     | Large documents, full chat history, large policy manuals.          | Large context feels safe but expensive and slower.                                 | Cost rises and answer quality may degrade from distraction.      | Retrieve only relevant sections, compress, deduplicate, hierarchical summaries.     | Requires better ingestion and retrieval design.                |
| RAG retrieval            | Vector search, metadata filtering, query rewriting, hybrid search. | Retrieval seems cheap until request volume and tenant count rise.                  | Vector DB costs, latency and infra load increase.                | Filter first, tune top-k, batch reads, optimize index, cache retrieval results.     | Too few chunks can reduce answer completeness.                 |
| Embedding generation     | Document ingestion, re-embedding, query embedding.                 | Teams forget recurring re-indexing and document churn.                             | Batch jobs become expensive overnight.                           | Incremental indexing, change detection, embedding cache, cheaper embedding model.   | Stale embeddings hurt retrieval freshness.                     |
| Vector DB storage/search | High-dimensional vectors, replicas, large metadata, high top-k.    | Storage and query units scale with corpus and tenants.                             | Search latency and DB bill rise after adding documents.          | Dimension choice, sharding, filters, index tuning, dedupe, lifecycle policies.      | Over-tuning can reduce recall.                                 |
| Reranking models         | Cross-encoder rerankers, LLM-based relevance scoring.              | Quality win hides latency/cost multiplier.                                         | One request triggers many reranker scores.                       | Rerank only ambiguous/high-stakes queries; reduce candidates; use lighter reranker. | Less reranking can lower precision.                            |
| Agent tool calls         | Planning, retrieval, CRM calls, web/API calls, reflection.         | One user request becomes 5-20 sub-requests.                                        | Cost/request varies wildly by agent path.                        | Max steps, tool budgets, allowlists, deterministic routing, cache tools.            | Too strict limits may block complex workflows.                 |
| Multi-step reasoning     | Chain decomposition, verification, critique, self-reflection.      | Looks intelligent in demos but expensive at scale.                                 | High output tokens and long traces.                              | Use only for complex/high-risk tasks; route simple tasks to direct answers.         | May reduce reliability on hard tasks.                          |
| Model retries            | Timeouts, parsing failures, provider errors, safety re-prompts.    | Retries are invisible to product metrics unless traced.                            | Cost spikes during incidents.                                    | Idempotency, retry budgets, backoff, schema validation, fallback models.            | Too few retries can reduce availability.                       |
| Streaming responses      | Long generations, connection overhead, UI session time.            | Streaming feels free but still generates tokens.                                   | Users ask follow-ups before completion; infra holds connections. | Stream only user-visible long generation; cancel unused streams.                    | Streaming hides but does not remove backend cost.              |
| Logging and tracing      | Full prompts, completions, embeddings, traces.                     | Necessary for debugging but storage costs grow.                                    | Observability bill and privacy risk increase.                    | Sample, redact, TTL policies, structured metrics over raw payloads.                 | Less detail may slow incident analysis.                        |
| Evaluation pipelines     | Regression tests, synthetic queries, judge models.                 | Offline evals can call expensive models repeatedly.                                | CI/CD or nightly jobs become large hidden spend.                 | Sample eval set, stratified eval, cheaper judges, run deltas only.                  | Weak eval coverage can miss quality regressions.               |
| Batch jobs               | Summarization, enrichment, embedding, migration.                   | Batch work is not visible to interactive users.                                    | Monthly bill spikes after data refresh.                          | Queue budgets, incremental processing, off-peak scheduling, rate limits.            | Longer processing windows.                                     |
| Fine-tuning              | Training data prep, training runs, hosted fine-tuned model.        | Teams focus on training price not lifecycle cost.                                  | Additional model version operations and evals.                   | Use when behavior repeats and prompts become large; compare with RAG/prompting.     | Can lock team into provider or stale behavior.                 |
| GPU/inference hosting    | Self-hosted models, replicas, idle capacity.                       | GPU cost continues even when traffic is low.                                       | Low utilization and expensive standby.                           | Autoscaling, batching, quantization, right-size model, managed API for low volume.  | Self-hosting adds operational complexity.                      |
| Third-party APIs         | Search, CRM, OCR, document parsing, rerank APIs.                   | LLM is not the only bill.                                                          | External API fees scale with agent/tool use.                     | Cache, batch, negotiate limits, deterministic pre-filtering.                        | Caching can introduce freshness and security issues.           |
| Human review             | Approval queues, compliance review, expert validation.             | Operational cost is often ignored in engineering estimates.                        | Support teams become bottleneck after rollout.                   | Risk-based routing, active learning, better UI, policy automation.                  | Automation must not bypass required controls.                  |

### Field note

When cost increases, do not ask only "Which model is expensive?" Ask: Which features, tenants, prompts, tools, retries, eval jobs, and batch workflows are consuming the budget? Cost allocation by request path is the difference between guessing and engineering.

### 3. Core Latency Drivers in GenAI Systems

Latency in GenAI systems is a chain. Improving one link does not help if another link dominates. A senior engineer decomposes response time into first-token latency, retrieval latency, tool latency, generation time, queueing, cold starts, and network overhead.

| Driver                     | Symptoms                                    | Root causes                                                     | Metrics to check                                     | Optimization strategy                                                           | Prevention pattern                                 |
|----------------------------|---------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------|
| Model inference time       | Long wait before or during generation.      | Large model, long prompt, long output, provider queueing.       | Model latency, tokens/sec, queue time, model tier.   | Compare model tiers, reduce prompt/output, route simple tasks to smaller model. | Use model routing; avoid large model as default.   |
| First-token latency        | UI appears frozen before streaming starts.  | Provider queueing, prompt size, safety checks, cold connection. | Time to first token, provider latency, request size. | Keep prompt lean, reuse connections, stream, warm paths.                        | Monitor first-token separately from total latency. |
| Total response time        | Answer finishes too slowly.                 | Long output, sequential calls, tool delays.                     | End-to-end duration, output tokens, tool trace.      | Limit output, parallelize safe work, async background tasks.                    | Use concise answer defaults.                       |
| Retrieval latency          | Slow before model call.                     | Filters, large index, poor metadata, network to vector DB.      | Retrieval duration, DB CPU, index stats, top-k.      | Filter before vector search, tune index, cache, reduce top-k.                   | Pre-test retrieval at target corpus size.          |
| Vector search latency      | Search worsens as documents grow.           | Unoptimized index, large dimensions, high recall settings.      | P95 query time, shard load, index memory.            | Index tuning, sharding, quantization, metadata pruning.                         | Separate hot/warm corpora.                         |
| Reranker latency           | Good answers but slow pipeline.             | Reranking too many candidates or using heavy model.             | Reranker duration, candidate count.                  | Rerank fewer docs, use only when needed, lighter reranker.                      | Gate reranking by query ambiguity.                 |
| Tool/API latency           | Agent waits on CRM, ERP, search, database.  | Synchronous external calls and slow dependencies.               | Per-tool latency, timeout rate, dependency SLA.      | Parallelize, cache, set timeouts, degrade gracefully.                           | Budget tool calls per workflow.                    |
| Agent planning loops       | Agent appears to think forever.             | Reflection loops, poor stopping conditions, repeated retrieval. | Step count, trace length, repeated tool calls.       | Max steps, deterministic routers, smaller planning model.                       | Every agent path needs a budget.                   |
| Cold starts                | First request after idle is slow.           | Serverless/container scale-to-zero, model warm-up.              | Cold start count, startup time, idle duration.       | Min replicas, warmers, keep-alive, provisioned concurrency.                     | Balance idle cost vs latency.                      |
| Network latency            | Regional users see slower responses.        | Long distance to provider, vector DB, APIs.                     | RTT, region, DNS, TLS handshake.                     | Regional deployment, connection pooling, colocate services.                     | Data residency may constrain regions.              |
| Database latency           | Permission checks or metadata lookup slow.  | Unindexed joins, tenant filters, audit writes.                  | DB query time, slow query logs, locks.               | Indexes, denormalized read models, async writes.                                | Do not bypass auth for speed.                      |
| Large prompt construction  | Backend slow before model call.             | Loading many docs, string assembly, serialization.              | Prompt build time, payload size.                     | Precompute, cache static parts, compress context.                               | Version prompt templates.                          |
| Long output generation     | Streaming starts but completion drags.      | Verbose format, no max token limit.                             | Output tokens, tokens/sec.                           | Short answer defaults, progressive details, max output.                         | Allow expansion on demand.                         |
| Sequential workflow design | Each step waits for previous unnecessarily. | Linear retrieval, tool calls, validation, logging.              | Trace waterfall, critical path.                      | Parallelize independent calls, prefetch, async non-critical work.               | Avoid parallelizing unsafe side effects.           |
| Synchronous external calls | Request times out due to dependency.        | Real-time wait for slow API.                                    | Dependency P95/P99, timeout count.                   | Queue async jobs, callbacks, optimistic UI.                                     | Use async for non-immediate tasks.                 |
| Auth and permission checks | Secure retrieval is slow.                   | Complex ACL joins, per-doc checks after search.                 | Permission filter time, DB queries/request.          | Precompute ACL metadata, filter before search, cache permissions carefully.     | Never leak data for speed.                         |
| Observability overhead     | Tracing/logging slows hot path.             | Full payload logging, sync telemetry writes.                    | Telemetry duration, log volume.                      | Async logging, sampling, redaction, metrics over raw traces.                    | Keep enough detail for incidents.                  |

#### Debugging approach

Use distributed tracing to create a latency waterfall. Mark auth, retrieval, reranking, prompt build, model first-token, generation, tool calls, post-processing, logging, and network return. Optimize the slowest component first, but check P95 and P99 rather than only averages.

## 4. Cost vs Latency vs Quality Trade-Off Framework

Optimization is not about making everything cheaper and faster at any cost. It is about choosing the minimum-cost and minimum-latency design that still meets task quality, safety, security, and user-experience requirements.

| Situation                                  | Optimization option                                                          | Benefit                                             | Risk                                          | When to use                                            | When to avoid                                              | Interview explanation                                                                                            |
|--------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------|-----------------------------------------------|--------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Smaller model vs larger model              | Route simple tasks to smaller model; reserve large model for reasoning.      | Lower cost and faster latency.                      | Lower accuracy on complex or ambiguous tasks. | Classification, extraction, routing, simple summaries. | High-risk legal, financial, medical, security reasoning.   | "I would match model strength to task risk and complexity instead of using the premium model for every request." |
| Fast answer vs accurate answer             | Use tiered response: quick draft then deeper verification.                   | Better perceived latency.                           | Initial answer may be incomplete.             | Operational chat, support triage, user guidance.       | Regulated or irreversible decisions.                       | "I would separate perceived latency from final confidence and make high-risk paths slower but safer."            |
| More retrieval context vs less context     | Tune top-k and context compression.                                          | Reduced tokens and faster model call.               | May miss supporting evidence.                 | Questions with narrow evidence need.                   | Broad legal/policy synthesis.                              | "More context is not always better; irrelevant context increases cost and can reduce answer quality."            |
| More chunks vs fewer chunks                | Use calibrated top-k by query type.                                          | Better cost/latency control.                        | Recall may drop.                              | Known-document lookup.                                 | Ambiguous exploratory queries.                             | "I would evaluate recall and answer faithfulness while reducing top-k."                                          |
| Reranking vs direct retrieval              | Rerank only when retrieval confidence is low.                                | Better precision on hard queries.                   | Extra model/API latency and cost.             | Ambiguous, high-value, enterprise search.              | Very low-latency chat path.                                | "Reranking should be a gated quality tool, not an unconditional tax."                                            |
| Agentic workflow vs deterministic workflow | Use deterministic routing for common paths; agent only for open-ended tasks. | Lower latency and predictable cost.                 | Less flexibility.                             | Known workflows, stable tools.                         | Ambiguous multi-step problem solving.                      | "I would not use an agent where a workflow engine or router is enough."                                          |
| Real-time vs async processing              | Move heavy document work to background jobs.                                 | Avoids timeouts and improves UX.                    | User waits for completion notification.       | Bulk summarization, embedding, report generation.      | Live conversation requiring immediate answer.              | "Not every AI workflow should be synchronous."                                                                   |
| Streaming vs non-streaming                 | Stream long user-facing outputs.                                             | Improves perceived latency.                         | Does not reduce total compute cost.           | Narrative answers, copilots.                           | Short classification/extraction tasks.                     | "Streaming is UX optimization, not a substitute for backend optimization."                                       |
| Fine-tuning vs prompt engineering          | Fine-tune repetitive behavior to reduce prompt length.                       | Shorter prompts, consistent style.                  | Training/eval/versioning cost.                | Repeated structured tasks with stable target style.    | Fast-changing knowledge or permissions.                    | "I would compare fine-tuning against RAG and prompt compression using production traffic."                       |
| Caching vs freshness                       | Cache stable and permission-safe results.                                    | Large latency and cost reduction.                   | Stale or unauthorized answers.                | FAQs, embeddings, tool results with TTL.               | User-specific sensitive answers with changing permissions. | "Caching must be permission-aware; speed is not worth cross-tenant leakage."                                     |
| Batch vs live inference                    | Batch non-interactive work.                                                  | Lower unit cost, controlled throughput.             | Delayed results.                              | Document ingestion, enrichment, evals.                 | Interactive support/chat.                                  | "Batching is ideal when user value does not require immediate response."                                         |
| Self-hosted vs managed API                 | Choose by volume, latency, compliance, ops maturity.                         | Control, data residency, potential scale economics. | Operational burden and GPU utilization risk.  | Predictable high volume or strict data controls.       | Low/variable traffic with small team.                      | "I would compare total cost of ownership, not only token or GPU price."                                          |

## 5. Model Selection Optimization

Model selection is one of the highest-leverage cost and latency decisions. A senior GenAI engineer does not ask, "What is the best model?" They ask, "What is the weakest reliable model for this specific task, risk level, latency target, and budget?"

### Model role patterns

| Model role          | Best use                                                     | Optimization logic                                                               | Risk control                                                       |
|---------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Routing model       | Intent classification, complexity scoring, policy detection. | Cheap and fast model decides whether premium reasoning is necessary.             | Add confidence threshold and fallback to stronger model.           |
| Extraction model    | Structured fields from tickets, PDFs, emails.                | Small model or constrained parser often enough.                                  | Validate with schema and deterministic checks.                     |
| Summarization model | Short summaries of documents, calls, tickets.                | Use cheaper model when factuality risk is moderate and source remains available. | Cite source chunks and limit abstraction.                          |
| Reasoning model     | Complex trade-off, legal reasoning, architecture design.     | Use only when task requires multi-step judgment.                                 | Gate by risk, audit traces, require human review if needed.        |
| Embedding model     | Semantic retrieval representation.                           | Choose by domain recall, dimension, latency, storage cost.                       | Evaluate retrieval against real queries, not benchmark names only. |
| Reranker model      | Ranking retrieved candidates.                                | Improve precision without sending all context to generator.                      | Use only for ambiguous or high-value queries.                      |
| Multimodal model    | Image/PDF/table understanding.                               | Powerful but expensive; use OCR/document parsing first where adequate.           | Route pages/regions instead of whole files.                        |

### Model cascading examples

| Pattern                                              | Example                                                                                                                     | Why it reduces cost/latency                            | Failure handling                                  |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------|
| Cheap -> medium -> premium                           | Classify request with small model; use medium model for normal support answers; premium model for complex policy conflicts. | Most traffic avoids the premium model.                 | Escalate when confidence low or policy risk high. |
| Rules before LLM                                     | Use deterministic regex/rules for invoice ID extraction before calling model.                                               | Avoids unnecessary inference for simple cases.         | Fallback to model when rule confidence low.       |
| Retrieval confidence gate                            | If top documents have high score and clean permissions, generate directly; otherwise query rewrite and rerank.              | Avoids reranking on easy queries.                      | Trigger rerank when top score margin is low.      |
| Summarize with cheap model, verify with strong model | Cheap model creates draft; strong model checks high-risk facts.                                                             | Premium model sees shorter content and only validates. | Use for compliance-heavy reports.                 |
| Provider fallback                                    | Primary provider for quality; fallback for availability/latency incidents.                                                  | Improves resilience and can control cost.              | Keep prompts and schemas provider-portable.       |

### Context window size vs cost

Large context windows are useful when the question genuinely requires broad evidence. They are dangerous when used as a substitute for retrieval quality. More context increases input cost, increases first-token latency, and can distract the model with irrelevant evidence. The optimization target is not maximum context; it is minimum sufficient context.

#### Interview-ready model selection answer

"I would build a model routing layer. Simple classification and extraction would go to a small low-latency model. Standard answers would use a mid-tier model with RAG. High-risk reasoning, ambiguous multi-document synthesis, or executive-facing outputs would escalate to a stronger model. I would log model choice, confidence, cost, latency, and outcome quality so routing decisions improve over time."

## 6. Prompt Optimization for Cost and Latency

Prompts are production artifacts. Prompt bloat is one of the most common hidden causes of cost increase and first-token latency. Every repeated instruction, unused example, stale policy block, hidden chat history entry, and uncompressed retrieved chunk becomes part of the bill.

### Prompt optimization principles

- Separate static instructions from dynamic request content so static prompt segments can be cached or versioned.
- Remove repeated instructions across system, developer, and user prompt layers.
- Use few-shot examples only when they measurably improve accuracy.
- Prefer structured output when downstream systems consume the answer, but avoid overly complex schemas that cause retries.
- Limit output length and provide expansion-on-demand patterns.
- Compress retrieved context with source identifiers and only necessary passages.
- Track prompt template version, token usage, error rate, and quality impact before rollout.

### Before/after production prompt example

#### Expensive verbose prompt

You are an extremely helpful enterprise AI assistant. Always answer with empathy, politeness, high detail, step-by-step reasoning, deep explanations, examples, caveats, a summary, and recommended next steps. The user may ask about policy documents. Read all provided context carefully and use it. Do not hallucinate. Follow all company policies. Be concise but also comprehensive. Here are 12 examples of ideal answers... [large few-shot block] ... Here is the full conversation history... Here are 20 retrieved chunks... Write a detailed answer.

#### Optimized production prompt

Role: Enterprise policy assistant.

Task: Answer the user's question using only the provided approved context.

Rules:

- Cite source\_ids for factual claims.
- Say "I do not have enough evidence" when context is insufficient.
- Keep answer under 180 words unless user asks for detail.
- Return: answer, source\_ids, confidence, missing\_info.

Context: {top\_relevant\_chunks\_compressed}

Question: {user\_question}

| Optimization                        | Effect                                                         |
|-------------------------------------|----------------------------------------------------------------|
| Removed repeated style instructions | Fewer input tokens and less instruction conflict.              |
| Reduced few-shot examples           | Lower prompt size; keep only examples that improve eval score. |
| Compressed context                  | Lower cost and faster first token.                             |
| Output limit                        | Lower output-token cost and faster completion.                 |
| Structured fields                   | Easier downstream parsing and observability.                   |

### Prompt rollout discipline

- Run prompt changes through regression evaluation before production.
- Compare token usage before and after prompt changes.
- Measure first-token latency, total latency, parse failures, retries, and user correction rate.
- Version every prompt template and link incidents to prompt versions.
- Have a rollback path for prompt changes that increase cost or reduce quality.

## 7. RAG Cost and Latency Optimization

RAG optimization is not simply vector database tuning. It is a full pipeline problem: document parsing, chunking, metadata, permissions, embedding, indexing, query rewriting, retrieval, reranking, context packing, answer generation, and evaluation.

| RAG lever                  | Trade-off                                                                                                              | Production guidance                                                                        |
|----------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Chunk size                 | Small chunks improve precision but may require more chunks; large chunks reduce calls but can include irrelevant text. | Tune by document type and question type; evaluate faithfulness and recall.                 |
| Chunk overlap              | Overlap prevents boundary loss but increases storage and duplicate context.                                            | Use minimal overlap; avoid blind 20% overlap on every corpus.                              |
| Metadata filtering         | Tenant, department, document type, date, ACL, region.                                                                  | Apply filters before vector search when possible to reduce search space and avoid leakage. |
| Permission-aware retrieval | Only retrieve documents the user can access.                                                                           | Precompute ACL metadata; never filter only after generation.                               |
| Hybrid search              | Combines keyword and semantic signals.                                                                                 | Use for enterprise terms, acronyms, IDs, product names, legal clauses.                     |
| Query rewriting            | Improves recall for ambiguous queries.                                                                                 | Gate it; rewriting every query adds latency and cost.                                      |
| Top-k tuning               | Controls number of retrieved chunks.                                                                                   | Measure answer quality as top-k decreases; top-k 20 may not beat top-k 5.                  |
| Context compression        | Summarize or extract only relevant lines.                                                                              | Useful when retrieval returns long chunks; must preserve citations.                        |
| Deduplication              | Remove repeated docs, version duplicates, boilerplate.                                                                 | Cuts storage, retrieval noise, token cost, and hallucination risk.                         |
| Embedding freshness        | New or changed docs need updated vectors.                                                                              | Use incremental indexing and content hashes.                                               |
| Vector DB tuning           | Index type, shards, replicas, dimensions, filtering strategy.                                                          | Tune using production-like corpus size and tenant filters.                                 |
| Retrieval evaluation       | Recall@k, MRR, answer faithfulness, citation correctness.                                                              | Do not optimize top-k blindly; evaluate real questions.                                    |

### Practical RAG optimization examples

| Situation               | Naive design                                  | Optimized design                                                      | Why it works                                                                    |
|-------------------------|-----------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Top-k too high          | Retrieve 20 chunks for every query.           | Use top-k 5 by default; escalate to 10 only when score margin is low. | Cuts prompt tokens and retrieval overhead while preserving most simple answers. |
| Tenant corpus too large | Vector search across all tenants then filter. | Apply tenant and ACL filters before search.                           | Reduces latency and prevents cross-tenant leakage.                              |
| Duplicate documents     | Index every upload and copy.                  | Use content hash, version policy, canonical documents.                | Improves retrieval precision and lowers storage.                                |
| Ambiguous queries       | Always call expensive reranker.               | Use reranker only when retrieval confidence low.                      | Keeps quality on hard queries without taxing easy ones.                         |
| Long policy manuals     | Send entire section to model.                 | Retrieve relevant passages and compress to facts with source IDs.     | Reduces tokens and improves focus.                                              |
| Frequent doc updates    | Full re-embedding every night.                | Incremental indexing with change detection.                           | Cuts batch cost and keeps freshness.                                            |

### RAG architecture note

- Ingestion path: parse -> normalize -> deduplicate -> chunk -> classify metadata -> compute permissions -> embed -> index -> validate retrieval.
- Query path: authenticate -> infer intent -> apply tenant/ACL filters -> retrieve -> optional rerank -> compress context -> generate -> cite -> log metrics.
- Evaluation path: sample real questions -> check recall, faithfulness, citation quality, latency, token cost, and escalation rate.

#### Senior RAG position

The best RAG optimization is often not a faster vector database. It is cleaner documents, better metadata, permission filters before search, fewer duplicate chunks, calibrated top-k, and a generator prompt that

only receives the minimum evidence needed.

## 8. Agent Cost and Latency Optimization

Agents become expensive because they convert one user request into an unpredictable workflow. Without budgets, stopping conditions, tool allowlists, and trace analysis, the agent can repeatedly plan, retrieve, call tools, reflect, and retry.

### Why agents become slow and expensive

- Too many planning steps before taking useful action.
- Tool call loops caused by ambiguous tool descriptions or missing state.
- Repeated reflection and self-critique even for simple tasks.
- Unbounded retries after parsing or tool failures.
- Large intermediate context carried across every step.
- Repeated retrieval instead of cached retrieval results.
- Serial tool execution when safe calls could run in parallel.
- External API delays and lack of timeout budgets.
- Agent memory growth that increases prompt cost every turn.

### Agent optimization patterns

| Pattern                       | How it works                                             | Benefit                                 | Risk control                                             |
|-------------------------------|----------------------------------------------------------|-----------------------------------------|----------------------------------------------------------|
| Hard max step limits          | Stop after N reasoning/tool steps.                       | Predictable latency and cost.           | Escalate to human or ask clarification if limit reached. |
| Tool allowlists               | Expose only relevant tools for request type.             | Reduces wrong calls and planning noise. | Route by intent first.                                   |
| Tool call budgeting           | Set max tool calls and max cost per request.             | Prevents runaway workflows.             | Record budget exhaustion as a trace event.               |
| Parallel tool calls           | Run independent read-only calls concurrently.            | Reduces wall-clock latency.             | Do not parallelize side-effecting actions blindly.       |
| Cached tool results           | Reuse CRM/profile/search results within request/session. | Cuts repeated dependency calls.         | Apply TTL and permission checks.                         |
| Deterministic routing         | Use rules/workflows for common paths.                    | Avoids agent overhead.                  | Fallback to agent for ambiguous tasks.                   |
| Smaller planning model        | Use cheaper model to choose tools.                       | Lowers cost and latency.                | Escalate if confidence low.                              |
| Summarized intermediate state | Carry compact state instead of full trace.               | Reduces prompt tokens.                  | Preserve audit log outside prompt.                       |
| Human approval                | Pause before expensive/sensitive actions.                | Controls risk and cost.                 | Use thresholds and clear user explanation.               |
| Trace analysis                | Analyze step patterns and loops.                         | Find root causes from real traffic.     | Add tests for repeated failure paths.                    |

### Detailed example: 45 seconds and €1.20 per request

| Stage         | Observation                                                                                         | Fix                                                                                        |
|---------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Problem       | Agentic CRM assistant takes 45 seconds and costs €1.20/request.                                     | Define request budget: target <10 seconds and <€0.25/request.                              |
| Root cause    | Planning model repeatedly calls retrieval and CRM lookup, then reflects after every tool result.    | Trace shows 9 model calls, 7 retrieval calls, 4 CRM calls, and large intermediate context. |
| Immediate fix | Set max 4 steps; cache retrieval and CRM result inside request; use smaller model for tool routing. | Request drops to 8 seconds and €0.18/request on common cases.                              |
| Long-term fix | Create deterministic routes for account lookup, ticket summary, and next-best-action.               | Agent only used for ambiguous multi-step requests.                                         |
| Prevention    | Agent trace dashboard tracks step count, tool count, repeated calls, cost/request, timeout rate.    | Alerts trigger when median step count or cost/request crosses threshold.                   |

### Interview line

"I would treat the agent as an execution graph with budgets, not as an open-ended reasoning loop. The key controls are max steps, tool budgets, deterministic routing, cached tool results, smaller planning models, and trace-based debugging."

## 9. Caching Strategies

Caching can produce the biggest cost and latency improvement in GenAI systems, but it can also create security incidents if implemented casually. Enterprise caching must be tenant-aware, permission-aware, version-aware, and freshness-aware.

| Cache type               | What it caches                                      | When useful                                            | Security/freshness rule                                                   |
|--------------------------|-----------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------|
| Prompt caching           | Static system instructions, shared policy blocks.   | Large repeated prompts with stable content.            | Do not cache user-specific sensitive context across users.                |
| Semantic caching         | Similar questions mapped to existing answers.       | FAQ-style support and public/internal knowledge bases. | Can return stale or mismatched answer if similarity threshold poor.       |
| Response caching         | Exact same request and context returns same answer. | Deterministic FAQs, low-risk informational answers.    | Include tenant, user permission, document version, prompt version in key. |
| Embedding caching        | Cache embeddings for unchanged text.                | Ingestion pipelines, repeated queries.                 | Use content hash and embedding model version.                             |
| Retrieval result caching | Cache top documents for repeated queries.           | Stable corpora and high-frequency questions.           | Invalidate on document, ACL, or metadata changes.                         |
| Tool result caching      | Cache CRM/account/product lookup results.           | Read-heavy tool calls with acceptable TTL.             | Do not cache sensitive dynamic data too broadly.                          |
| Session-level caching    | Reuse context in a user session.                    | Conversational workflows.                              | Clear at logout/session expiry.                                           |
| Tenant-level caching     | Shared docs/config within one customer tenant.      | Large enterprise tenants with repeated queries.        | Never mix tenant keys.                                                    |
| CDN/API gateway caching  | Static resources, model-independent metadata.       | Documentation portals and static results.              | Avoid dynamic personalized responses.                                     |

### What can be cached safely

- Public or tenant-approved static documentation with version keys.
- Embeddings keyed by content hash and embedding model version.
- Prompt templates keyed by prompt version.
- Low-risk tool results with short TTL and permission-bound cache keys.
- Retrieval results only when tenant, user permissions, document versions, and filters are part of the cache key.

### What should never be cached casually

- Cross-tenant responses without tenant isolation.
- Sensitive user-specific answers without explicit permission-bound keys.
- Results based on permissions that can change quickly unless invalidation is strong.
- Regulated data where retention policy prohibits storing derived outputs.
- Tool responses containing secrets, access tokens, credentials, or private notes.

#### Cache key rule

A safe enterprise cache key often includes: tenant\_id + user\_id or permission\_signature + document\_version + prompt\_version + model\_version + request\_normalized\_hash + tool\_state\_version. If this feels too heavy, that is a sign the answer may not be safe to cache broadly.

## 10. Streaming and User Experience Optimization

Streaming improves perceived latency by showing progress early. It does not automatically reduce total latency or cost. A good GenAI product uses streaming together with backend optimization, progressive disclosure, cancellation, and clear loading states.

| UX concept             | Meaning                                      | Optimization tactic                                                                   |
|------------------------|----------------------------------------------|---------------------------------------------------------------------------------------|
| First-token latency    | Time before user sees first output.          | Stream early, reduce prompt size, warm connections, avoid pre-generation bottlenecks. |
| Total latency          | Time until final answer is complete.         | Limit output length, parallelize safe steps, reduce tool calls.                       |
| Perceived latency      | How slow the user feels the system is.       | Streaming, progress states, partial results, skeleton UI.                             |
| Progressive disclosure | Give short answer first, details on request. | Default concise answer; add "show details" expansion.                                 |
| Async job handling     | Long-running tasks move out of request path. | Return job status, notification, downloadable result.                                 |
| Cancellation support   | User can stop long generation.               | Cancel model stream, stop tool execution, log cancelled cost.                         |
| Retry UX               | User understands when a dependency failed.   | Retry safely, provide partial answer and next step.                                   |
| Timeout handling       | System avoids endless wait.                  | Set per-component and global timeout budgets.                                         |

### When streaming helps

- Long narrative or advisory answers where partial text is useful.
- Assistant-style interfaces where users expect conversational flow.
- Complex answer generation where first-token latency can be kept low.
- Demos where perceived responsiveness matters.

### When streaming hides deeper problems

- Retrieval or tool calls take 10 seconds before any token starts.
- The answer is too verbose and should be shorter.
- Agent loops keep running after user no longer needs the result.
- The UI streams low-confidence content before verification completes.

## 11. Infrastructure and Deployment Optimization

Infrastructure optimization for GenAI systems is about controlling the critical path. A production deployment must manage connection reuse, queueing, concurrency, backpressure, autoscaling, cold starts, regional latency, GPU utilization, and observability overhead.

| Infrastructure lever     | Optimization use                                                       | Production warning                                      |
|--------------------------|------------------------------------------------------------------------|---------------------------------------------------------|
| API gateway              | Rate limits, auth offload, request shaping, caching headers.           | Do not hide per-tenant cost attribution.                |
| Connection pooling       | Reuse HTTP/database connections to providers and tools.                | Pool exhaustion creates latency spikes.                 |
| HTTP keep-alive          | Avoid repeated TLS handshakes.                                         | Tune timeouts and idle connection limits.               |
| Async processing         | Move long-running jobs to queues.                                      | Users need job status and failure handling.             |
| Worker scaling           | Scale background workers by queue depth and SLA.                       | Avoid uncontrolled batch spend.                         |
| Horizontal scaling       | Add replicas for stateless API layers.                                 | Model/provider latency may still dominate.              |
| Rate limiting            | Protect system and budget.                                             | Make limits tenant-aware and user-friendly.             |
| Backpressure             | Slow down producers before overload.                                   | Prefer graceful degradation over cascading failure.     |
| Autoscaling              | Scale by CPU, queue depth, concurrency, latency.                       | LLM workloads may not correlate with CPU alone.         |
| Cold start reduction     | Min replicas, warmers, provisioned concurrency.                        | Costs more idle capacity.                               |
| Regional deployment      | Deploy near users/providers/data.                                      | Data residency and vendor region constraints matter.    |
| GPU optimization         | Batching, quantization, tensor parallelism, memory tuning.             | Low utilization can be more expensive than managed API. |
| Serverless vs containers | Serverless for bursty low-volume APIs; containers for stable services. | Cold starts and timeout limits affect GenAI workflows.  |
| Kubernetes tuning        | Requests/limits, HPA, node pools, pod disruption budgets.              | Over/under-provisioning creates cost or instability.    |
| Batch inference          | Group offline tasks for throughput.                                    | Not suitable for interactive low-latency tasks.         |
| Observability overhead   | Async traces, metrics sampling, payload redaction.                     | Full prompt logging can be costly and risky.            |

### Architecture patterns

| Pattern                            | Use case                               | Key optimization decisions                                              |
|------------------------------------|----------------------------------------|-------------------------------------------------------------------------|
| Real-time GenAI API                | Chatbot, copilots, sales assistant.    | Keep critical path short; route models; stream; cache; strict timeouts. |
| Async document processing pipeline | Summarization, extraction, embedding.  | Queue, batch, worker autoscaling, cost budgets, retry policies.         |
| RAG ingestion pipeline             | Enterprise document search assistant.  | Deduplication, incremental indexing, ACL metadata, embedding cache.     |
| Multi-tenant GenAI platform        | Customer-facing SaaS AI features.      | Tenant budgets, isolation, per-tenant metrics, permission-aware cache.  |
| Agent execution service            | CRM/ERP assistant, operations copilot. | Step limits, tool budgets, trace store, deterministic routes.           |
| Evaluation pipeline                | Prompt/model regression testing.       | Sampled evals, cheaper judge models, scheduled/offline execution.       |

## 12. Observability for Cost and Latency

A production GenAI system without cost and latency observability is financially unsafe. The system must expose metrics at the level of request path, model, tenant, user segment, feature, prompt version, retrieval configuration, and agent trace.

| Metric                       | Definition                                    | Why it matters                              |
|------------------------------|-----------------------------------------------|---------------------------------------------|
| Request latency              | End-to-end response time.                     | Overall user experience and SLA.            |
| First-token latency          | Time before stream begins.                    | Perceived responsiveness.                   |
| Total generation time        | Time spent generating output.                 | Model/output bottlenecks.                   |
| Input/output tokens          | Tokens consumed per request.                  | Cost driver diagnosis.                      |
| Cost per request             | Estimated total request cost.                 | Budget and ROI tracking.                    |
| Cost per tenant/feature/user | Attribution by customer or product area.      | Billing, quotas, procurement conversations. |
| Retrieval/reranker latency   | RAG component timing.                         | Find non-model bottlenecks.                 |
| Tool call latency            | Per external dependency timing.               | Agent/API bottlenecks.                      |
| Agent step count             | Number of planning/tool steps.                | Loop and budget control.                    |
| Cache hit rate               | Share served from cache.                      | Cost/latency optimization health.           |
| Retry/timeout rate           | Failures and repeated attempts.               | Hidden cost multiplier.                     |
| P95/P99 latency              | Tail latency.                                 | Real production reliability.                |
| Provider/model latency       | Latency by vendor/model.                      | Fallback and routing decisions.             |
| Vector DB latency            | Search/index performance.                     | RAG scaling health.                         |
| Queue depth                  | Backlog in async pipelines.                   | SLA risk and scaling trigger.               |
| GPU utilization              | Self-hosted inference efficiency.             | Cost efficiency of capacity.                |
| Customer ROI metrics         | Resolution rate, hours saved, revenue impact. | Executive value story.                      |

### Dashboard sections

| Dashboard                     | Audience                        | Typical panels                                                                       |
|-------------------------------|---------------------------------|--------------------------------------------------------------------------------------|
| Executive cost dashboard      | CFO, sponsor, product owner.    | Spend trend, cost/request, cost/tenant, adoption, ROI proxy, budget alerts.          |
| Engineering latency dashboard | Platform and backend engineers. | P50/P95/P99, first-token, retrieval, model, tool, DB, queue, provider latency.       |
| Tenant usage dashboard        | Customer success, account team. | Requests, tokens, cost, active users, top workflows, budget remaining.               |
| Incident dashboard            | On-call, SRE, FDE.              | Errors, retries, timeouts, provider health, cache drop, queue depth, recent deploys. |
| Model performance dashboard   | AI engineers, product.          | Quality evals, model route distribution, fallback rate, cost/quality trade-off.      |

#### Trace event recommendation

Each request trace should include: tenant\_id, feature, prompt\_version, model, input\_tokens, output\_tokens, estimated\_cost, latency breakdown, retrieval top-k, reranker used, cache status, tool calls, retries, timeout status, and final outcome. Redact sensitive payloads and apply retention policy.

## 13. Cost Budgeting and Guardrails

Guardrails prevent runaway cost before it becomes a CFO escalation. A serious GenAI platform treats budget as a first-class engineering constraint, not as a monthly surprise.

| Guardrail                  | Purpose                                      | Implementation                                                 |
|----------------------------|----------------------------------------------|----------------------------------------------------------------|
| Per-user budget            | Limit individual misuse or accidental loops. | Daily/monthly token or spend cap with friendly warning.        |
| Per-tenant budget          | Contract-level predictability.               | Tenant dashboard and alert thresholds.                         |
| Per-feature budget         | Find expensive product areas.                | Budget by workflow: chat, summarization, agent, eval.          |
| Per-request token limit    | Prevent huge prompts/outputs.                | Max input context and output length.                           |
| Max agent steps/tool calls | Stop runaway agents.                         | Hard limit plus escalation path.                               |
| Rate limits and quotas     | Protect infra and budget.                    | Tenant-aware limits by tier.                                   |
| Spend anomaly detection    | Detect sudden cost increases.                | Alert on unusual token/request/cost pattern.                   |
| Expensive query detection  | Identify heavy documents or workflows.       | Flag long context, repeated retries, high tool usage.          |
| Model fallback rules       | Control provider latency/cost incidents.     | Fallback to cheaper/faster model under defined conditions.     |
| Human approval             | Gate high-cost/high-risk tasks.              | Approval before mass email, bulk processing, external actions. |

### Sample policies

| Policy                             | Typical rules                                                                                   | Customer-facing explanation                                            |
|------------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Free-tier policy                   | Small model default, strict daily quota, no long-context agent workflows, limited batch jobs.   | Keeps free usage sustainable while allowing product evaluation.        |
| Enterprise tenant policy           | Contract budget, quota alerts, premium model only for approved workflows, monthly usage report. | Provides predictability and supports procurement.                      |
| Internal employee assistant policy | Department budgets, sensitive workflows require approval, logging with retention limits.        | Encourages adoption without uncontrolled internal spend.               |
| High-risk workflow policy          | Strong model, retrieval citations, human approval, full audit trace, no broad cache.            | Optimizes for correctness and compliance over speed.                   |
| Demo environment policy            | Warm paths, seeded cache, capped traffic, fixed dataset, budget alert.                          | Ensures demo reliability without pretending it is production capacity. |

## 14. Enterprise Customer Communication

Cost and latency conversations are sensitive because they touch budget, trust, adoption, and executive expectations. The FDE should communicate with clarity, avoid blame, and translate technical bottlenecks into customer impact and options.

| Situation                             | Senior wording template                                                                                                                                                                                                                                                   |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Explaining high latency               | "I would not frame this as simply a model problem. I would break the latency into retrieval time, model inference time, tool-call time, and post-processing time. That lets us optimize the real bottleneck instead of guessing."                                         |
| Explaining cost increase              | "The increase appears to be driven less by user count and more by request shape: longer prompts, more retrieved context, additional tool calls, and retries. I would separate these drivers so we can reduce spend without reducing useful quality."                      |
| Executive trade-off                   | "We can make this faster and cheaper, but we need to decide which quality bar must remain fixed. For low-risk requests, we can route to a smaller model. For high-risk workflows, I recommend keeping the stronger model and optimizing retrieval and prompt size first." |
| Model downgrade decision              | "This is not a downgrade of the product experience. It is a routing strategy: use the premium model where it changes the outcome, and use a faster model where the task is simple and measurable."                                                                        |
| Why caching helps                     | "Many requests repeat the same stable context. A permission-aware cache can reduce latency and cost while preserving tenant isolation and freshness rules."                                                                                                               |
| Why not every request uses best model | "The best model is not always the best system choice. The right system uses the cheapest reliable model for the task and escalates only when complexity or risk requires it."                                                                                             |
| Optimization roadmap                  | "I would propose a three-step roadmap: first instrument the bottlenecks, then apply safe quick wins like prompt trimming and top-k tuning, and finally redesign heavy workflows with routing, caching, and async processing."                                             |
| Handling complaints                   | "Your concern is valid. I would separate whether the issue is answer quality, latency, cost predictability, or workflow fit. Each has a different fix, and treating everything as a model problem would be too imprecise."                                                |
| Failed pilot due to latency           | "The pilot showed useful quality but the workflow latency was not production-ready. I would treat this as an engineering optimization issue, not a failure of the entire AI use case."                                                                                    |
| CFO/procurement concern               | "We can provide per-tenant budgets, cost ceilings, usage dashboards, and model-routing rules so spend is predictable rather than open-ended."                                                                                                                             |
| Security concern about caching        | "Caching is only acceptable when the cache key includes tenant and permission boundaries, and when invalidation respects document and access changes. We should not cache sensitive cross-user answers broadly."                                                          |

### Customer-facing optimization narrative

1. Acknowledge the business concern: slow, costly, unpredictable, or hard to justify.
2. Decompose the system into measurable components.
3. Identify quick wins that do not compromise security or quality.
4. Explain trade-offs with options, not absolutes.
5. Connect every optimization to user adoption, ROI, budget predictability, or risk reduction.
6. Commit to measured follow-up: dashboards, thresholds, and before/after numbers.

## 15. Production Troubleshooting Scenarios

The following playbooks are designed for interviews and real incidents. The goal is not to memorize fixes, but to demonstrate a senior debugging sequence: clarify, measure, isolate, mitigate, prevent, and communicate.

### LLM cost increased 5x after rollout

| Playbook step          | Field guidance                                                                                                                                                  |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Daily spend increased sharply while active users increased only modestly.                                                                                       |
| Clarifying questions   | Which tenant, feature, prompt version, model route, retry path, batch job, or agent workflow changed?                                                           |
| Probable causes        | Longer prompts, new RAG context, agent loop, eval job, retries, output verbosity, provider price tier change.                                                   |
| Metrics to check       | Cost/request, tokens/request, requests by feature, retry count, model distribution, output tokens, eval/batch spend.                                            |
| Debugging steps        | Break spend by tenant/feature/model/prompt version; compare before/after rollout; inspect high-cost traces.                                                     |
| Immediate fix          | Apply max output length, pause expensive batch/eval jobs, route simple tasks to smaller model, cap agent steps.                                                 |
| Long-term fix          | Budgeting, anomaly alerts, prompt token regression tests, request-path cost dashboard.                                                                          |
| Prevention             | Set pre-rollout cost simulation and per-tenant budget alerts.                                                                                                   |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric.                                    |
| Interview-ready answer | "The increase is likely from request shape, not just user count. I would isolate token growth, retries, agent steps, and batch jobs before changing the model." |

### Latency jumped from 2 seconds to 20 seconds

| Playbook step          | Field guidance                                                                                                                    |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Users report the assistant is unusably slow after a deployment.                                                                   |
| Clarifying questions   | Did retrieval, reranker, prompt version, model, provider, tool dependency, or region change?                                      |
| Probable causes        | New reranker, larger context, provider latency incident, cold starts, tool timeout, synchronous logging.                          |
| Metrics to check       | P95/P99, first-token, retrieval, rerank, model latency, tool latency, timeout and retry rates.                                    |
| Debugging steps        | Inspect trace waterfall and compare component timing before/after deploy.                                                         |
| Immediate fix          | Rollback prompt/reranker change, disable optional tool, reduce top-k, activate fallback provider.                                 |
| Long-term fix          | Latency budgets per component, canary deploys, provider health monitoring.                                                        |
| Prevention             | Performance tests with realistic payload and corpus size.                                                                         |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric.      |
| Interview-ready answer | "I would not guess. I would use traces to see whether the 18-second increase is retrieval, model, tool calls, or infrastructure." |

### RAG retrieval became slow after adding more documents

| Playbook step          | Field guidance                                                                                                                                           |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Vector search latency grows after corpus expansion.                                                                                                      |
| Clarifying questions   | How many documents, chunks, dimensions, tenants, filters, and replicas changed?                                                                          |
| Probable causes        | Index not tuned, too many chunks, missing metadata filters, high top-k, hot shard.                                                                       |
| Metrics to check       | Vector DB P95, top-k, filter selectivity, index memory, shard load, corpus size.                                                                         |
| Debugging steps        | Replay representative queries and compare index/filter behavior.                                                                                         |
| Immediate fix          | Reduce top-k, add metadata filters, tune index, scale vector DB, deduplicate chunks.                                                                     |
| Long-term fix          | Capacity planning by corpus size and tenant; ingestion quality gates.                                                                                    |
| Prevention             | Benchmark retrieval at projected production corpus size.                                                                                                 |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric.                             |
| Interview-ready answer | "Adding documents changes retrieval economics. I would filter earlier, tune index settings, and remove duplicate chunks before simply scaling hardware." |

## Agent keeps calling tools repeatedly

| Playbook step          | Field guidance                                                                                                                            |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Agent repeats CRM lookup, retrieval, or calculator calls and times out.                                                                   |
| Clarifying questions   | Which tool repeats, what observation triggers it, and is state preserved?                                                                 |
| Probable causes        | Poor stopping condition, ambiguous tool descriptions, missing memory, failed parse, no result cache.                                      |
| Metrics to check       | Agent steps, tool count, repeated tool args, parse failures, timeout rate.                                                                |
| Debugging steps        | Inspect trace; identify loop trigger and missing state transition.                                                                        |
| Immediate fix          | Set max steps, cache tool result, improve tool schema, add deterministic route.                                                           |
| Long-term fix          | Agent budget tests and trace-based loop detection.                                                                                        |
| Prevention             | Unit tests for tool selection and loop prevention.                                                                                        |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric.              |
| Interview-ready answer | "I would treat this as a control-loop bug. The fix is step budgets, better tool routing, cached results, and explicit stopping criteria." |

## One tenant has unusually high usage

| Playbook step          | Field guidance                                                                                                                                           |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | One customer consumes far more tokens or requests than comparable tenants.                                                                               |
| Clarifying questions   | Is this legitimate adoption, abuse, a loop, batch job, or integration bug?                                                                               |
| Probable causes        | Automated integration loop, heavy document processing, power-user workflow, missing quota.                                                               |
| Metrics to check       | Cost/tenant, cost/user, request source, token/request, batch jobs, agent steps.                                                                          |
| Debugging steps        | Segment usage by API key, user, feature, time of day, and prompt version.                                                                                |
| Immediate fix          | Apply tenant rate limit, contact customer, pause suspicious workflow if necessary.                                                                       |
| Long-term fix          | Tenant budgets, usage dashboard, anomaly alerts, integration safeguards.                                                                                 |
| Prevention             | Contract-aligned quotas and notifications.                                                                                                               |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric.                             |
| Interview-ready answer | "I would first distinguish healthy adoption from runaway automation. The response differs: support scale-up for healthy usage, throttle loops or abuse." |

## Executive demo is too slow

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Demo answer takes too long in front of stakeholders.                                                                         |
| Clarifying questions   | Is the demo path hitting cold starts, large documents, slow provider, or live tools?                                         |
| Probable causes        | Unwarmed services, live external dependencies, over-large prompt, no streaming, poor network.                                |
| Metrics to check       | First-token latency, retrieval latency, model latency, tool calls, cold start count.                                         |
| Debugging steps        | Run rehearsal traces; identify demo-specific critical path.                                                                  |
| Immediate fix          | Warm services, pre-index documents, stream output, use stable demo dataset, shorten answer.                                  |
| Long-term fix          | Demo readiness checklist with latency rehearsal and fallback path.                                                           |
| Prevention             | Separate demo environment from production claims.                                                                            |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "For an executive demo, I would reduce live uncertainty while staying honest about production architecture and limitations." |

## Customer complains the AI is too expensive

| Playbook step          | Field guidance                                                                                                                |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Customer questions invoice or projected budget.                                                                               |
| Clarifying questions   | Which workflows drive cost, and what business value do they produce?                                                          |
| Probable causes        | Premium model overuse, verbose answers, no quotas, high batch jobs, low-value workflows.                                      |
| Metrics to check       | Cost by feature, active users, cost/outcome, model distribution, token/request.                                               |
| Debugging steps        | Create cost attribution report and identify low-value high-cost paths.                                                        |
| Immediate fix          | Route simple tasks to cheaper model, add quotas, trim prompts, cache safe repeated results.                                   |
| Long-term fix          | ROI dashboard and budget policies.                                                                                            |
| Prevention             | Cost review before expansion.                                                                                                 |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric.  |
| Interview-ready answer | "I would not defend the bill generically. I would show cost by workflow and propose concrete reductions that preserve value." |

## Vector database query latency increased

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | RAG latency rises while model latency unchanged.                                                                             |
| Clarifying questions   | Did index size, metadata filters, shard distribution, or traffic change?                                                     |
| Probable causes        | Index fragmentation, hot tenant, inefficient filters, under-provisioned replicas.                                            |
| Metrics to check       | Query latency, filter selectivity, QPS, shard CPU/memory, cache hit rate.                                                    |
| Debugging steps        | Compare slow and fast queries; inspect filters and index statistics.                                                         |
| Immediate fix          | Add selective filters, scale replicas, tune index, reduce dimensions/top-k.                                                  |
| Long-term fix          | Vector DB capacity planning and index maintenance.                                                                           |
| Prevention             | Load test with realistic tenant distribution.                                                                                |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "When vector DB latency rises, I check corpus growth, filter selectivity, top-k, shard load, and tenant hot spots."          |

## Reranker improved quality but doubled latency

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Answer quality improved but user experience worsened.                                                                        |
| Clarifying questions   | Which queries truly need reranking and how many candidates are scored?                                                       |
| Probable causes        | Reranking every query, too many candidates, heavy reranker model.                                                            |
| Metrics to check       | Reranker latency, candidate count, quality lift by segment.                                                                  |
| Debugging steps        | Run ablation by query type: no rerank vs rerank top 10 vs top 50.                                                            |
| Immediate fix          | Gate reranking for ambiguous/high-value queries; reduce candidates.                                                          |
| Long-term fix          | Quality-latency policy and retrieval confidence thresholds.                                                                  |
| Prevention             | Monitor quality lift per millisecond added.                                                                                  |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "Reranking should be conditional. If it improves only 20% of queries, do not tax 100% of traffic."                           |

## Prompt update increased token usage

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Cost and first-token latency rise after prompt release.                                                                      |
| Clarifying questions   | What changed in system prompt, few-shot examples, context packing, or output format?                                         |
| Probable causes        | Verbose instructions, duplicated rules, added examples, longer output default.                                               |
| Metrics to check       | Input tokens, output tokens, prompt version, parse errors, retry count.                                                      |
| Debugging steps        | Diff prompt templates and compare token metrics by version.                                                                  |
| Immediate fix          | Rollback or trim prompt; reduce examples; enforce output max.                                                                |
| Long-term fix          | Prompt CI checks for token budget and quality regression.                                                                    |
| Prevention             | Every prompt rollout includes cost impact report.                                                                            |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "Prompt changes are production changes. I would version them and compare token usage and latency before rollout."            |

## Model provider latency became unstable

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Latency spikes intermittently without application deploy.                                                                    |
| Clarifying questions   | Is instability model-specific, region-specific, or provider-wide?                                                            |
| Probable causes        | Provider incident, regional queueing, rate limiting, network issue.                                                          |
| Metrics to check       | Provider latency by model/region, fallback rate, error codes, queue time.                                                    |
| Debugging steps        | Compare providers/models and application traces during spike windows.                                                        |
| Immediate fix          | Activate fallback model/provider, reduce concurrency, degrade optional features.                                             |
| Long-term fix          | Provider health dashboard and multi-provider fallback strategy.                                                              |
| Prevention             | Test fallback prompts and schema compatibility.                                                                              |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "I would isolate provider latency from our own pipeline and use routing/fallback rules rather than waiting blindly."         |

## Cache hit rate dropped suddenly

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Latency and cost rise because fewer requests hit cache.                                                                      |
| Clarifying questions   | Did cache key, TTL, prompt version, document version, or traffic mix change?                                                 |
| Probable causes        | New prompt version invalidated cache, TTL too short, normalization changed, document churn.                                  |
| Metrics to check       | Cache hit rate by type, invalidations, TTL, key cardinality, traffic mix.                                                    |
| Debugging steps        | Compare cache keys before/after deploy; inspect invalidation events.                                                         |
| Immediate fix          | Rollback key change, adjust TTL, normalize queries, prewarm safe cache.                                                      |
| Long-term fix          | Cache observability and deployment checks.                                                                                   |
| Prevention             | Cache changes require performance and security review.                                                                       |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "A cache hit drop is usually a keying, versioning, traffic, or invalidation issue; I would inspect those before scaling."    |

## Batch embedding job became too expensive

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Nightly embedding bill spikes after document refresh.                                                                        |
| Clarifying questions   | Are all documents re-embedded or only changed content?                                                                       |
| Probable causes        | Full re-indexing, duplicate docs, no content hash, larger embedding model.                                                   |
| Metrics to check       | Embeddings generated, documents changed, duplicate rate, model version, job retries.                                         |
| Debugging steps        | Compare changed document count with embedded document count.                                                                 |
| Immediate fix          | Pause full job; switch to incremental indexing; deduplicate.                                                                 |
| Long-term fix          | Content-hash based embedding cache and change detection.                                                                     |
| Prevention             | Indexing budget and anomaly alerts.                                                                                          |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "Embedding should be incremental. Re-embedding unchanged documents is usually a pipeline design issue."                      |

## Long context window caused poor performance

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | Cost and latency high; answers become less focused.                                                                          |
| Clarifying questions   | Why is long context used: broad synthesis, weak retrieval, or convenience?                                                   |
| Probable causes        | Too much irrelevant context, chat history bloat, full documents in prompt.                                                   |
| Metrics to check       | Input tokens, answer accuracy, citation precision, first-token latency.                                                      |
| Debugging steps        | Compare long-context answer vs compressed RAG answer on eval set.                                                            |
| Immediate fix          | Use retrieval, summarization, context compression, history summarization.                                                    |
| Long-term fix          | Context budget policies per workflow.                                                                                        |
| Prevention             | Treat context size as a quality and cost variable.                                                                           |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "Long context is a tool, not a strategy. I would test whether smaller evidence improves both speed and faithfulness."        |

## Tool-calling workflow times out

| Playbook step          | Field guidance                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Symptoms               | LLM response fails because external workflow exceeds timeout.                                                                |
| Clarifying questions   | Which tool is slow, can it be async, and is the result required immediately?                                                 |
| Probable causes        | Slow external API, serial calls, no timeout budget, synchronous side effects.                                                |
| Metrics to check       | Tool latency, timeout rate, dependency errors, step count, retries.                                                          |
| Debugging steps        | Trace tool calls and identify critical-path dependency.                                                                      |
| Immediate fix          | Set tool timeouts, parallelize read-only calls, return partial answer or async job.                                          |
| Long-term fix          | Dependency SLAs and circuit breakers.                                                                                        |
| Prevention             | Workflow design review before adding tools.                                                                                  |
| Customer communication | Acknowledge impact, show measured cause, provide immediate mitigation, then explain long-term prevention and success metric. |
| Interview-ready answer | "A tool timeout is not always an LLM issue. I would redesign slow side effects as async and keep the user informed."         |

## 16. Interview Case Studies

### Build an enterprise legal RAG assistant under strict cost limits

| Case component                 | Content                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interviewer prompt             | A legal team wants a RAG assistant over contracts and policies. Budget is strict, answers must cite sources, and latency target is under 8 seconds.                                                                                                                                                                                                                                                     |
| Candidate clarifying questions | <ul style="list-style-type: none"> <li>- Who are the users: lawyers, sales, procurement, or executives?</li> <li>- What documents are authoritative and how are permissions represented?</li> <li>- What is the acceptable risk level for legal interpretation?</li> <li>- What latency and cost ceilings are contractual?</li> <li>- Is human review required for external-facing language?</li> </ul> |
| Weak answer                    | Use the strongest model with all documents in a long context window for every request.                                                                                                                                                                                                                                                                                                                  |
| Strong answer                  | Use permission-aware RAG with metadata filters, calibrated top-k, citation enforcement, and model routing. Simple clause lookup uses mid-tier model; high-risk synthesis escalates. Reranking only for ambiguous queries.                                                                                                                                                                               |
| Architecture reasoning         | Authentication -> tenant/ACL filters -> hybrid retrieval -> optional rerank -> compressed evidence -> answer with citations -> audit log.                                                                                                                                                                                                                                                               |
| Cost/latency trade-offs        | Cost limit requires top-k tuning and routing; legal quality requires citations and escalation; latency target discourages unconditional heavy reranking.                                                                                                                                                                                                                                                |
| Metrics to monitor             | Recall@k, citation correctness, cost/request, retrieval latency, model latency, human escalation rate.                                                                                                                                                                                                                                                                                                  |
| Final recommendation           | Pilot with limited document set, define high-risk categories, instrument cost and latency from day one.                                                                                                                                                                                                                                                                                                 |

### Optimize a customer support chatbot with 10,000 daily users

| Case component                 | Content                                                                                                                                                                                                                                                                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interviewer prompt             | Support chatbot volume is growing and monthly LLM spend is rising faster than ticket deflection.                                                                                                                                                                               |
| Candidate clarifying questions | <ul style="list-style-type: none"> <li>- Which intents dominate traffic?</li> <li>- What percentage are repeated FAQs?</li> <li>- What is the target cost per resolved issue?</li> <li>- Can answers be cached?</li> <li>- Which answers require live account data?</li> </ul> |
| Weak answer                    | Keep same model and hope caching at the browser layer helps.                                                                                                                                                                                                                   |
| Strong answer                  | Build intent router, cache safe FAQ answers, use small model for classification, RAG for knowledge answers, tool calls only for account-specific requests, concise response defaults.                                                                                          |
| Architecture reasoning         | Gateway -> intent classifier -> FAQ semantic cache -> RAG/tool path -> model route -> streaming answer.                                                                                                                                                                        |
| Cost/latency trade-offs        | Caching improves cost but must be permission-aware; smaller model helps simple traffic; live account data cannot be cached broadly.                                                                                                                                            |
| Metrics to monitor             | Cost/resolved ticket, deflection rate, cache hit rate, P95 latency, tool latency, CSAT.                                                                                                                                                                                        |
| Final recommendation           | Reduce cost by workflow, not across the board; protect high-value paths.                                                                                                                                                                                                       |

### Reduce latency for an agentic CRM assistant

| Case component                 | Content                                                                                                                                                                                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interviewer prompt             | Sales users complain the CRM assistant takes 30-45 seconds for account preparation.                                                                                                                                                                               |
| Candidate clarifying questions | <ul style="list-style-type: none"> <li>- What tools are called?</li> <li>- Are tool calls serial or parallel?</li> <li>- How many agent steps occur?</li> <li>- Which data is required synchronously?</li> <li>- Can account snapshots be precomputed?</li> </ul> |
| Weak answer                    | Switch to a faster model without inspecting agent traces.                                                                                                                                                                                                         |
| Strong answer                  | Trace the agent, cap steps, cache account data, parallelize read-only calls, precompute account summaries, route common requests to deterministic workflows.                                                                                                      |
| Architecture reasoning         | User -> deterministic intent route -> account cache -> parallel CRM/tool calls -> summarization -> optional agent for ambiguous next steps.                                                                                                                       |
| Cost/latency trade-offs        | Agent flexibility is useful but common CRM operations should be deterministic and cache-backed.                                                                                                                                                                   |
| Metrics to monitor             | Agent step count, tool latency, cache hit rate, timeout rate, cost/request.                                                                                                                                                                                       |
| Final recommendation           | Replace open-ended agent loop with bounded execution graph.                                                                                                                                                                                                       |

## Design a multi-tenant GenAI platform with per-customer budgets

| Case component                 | Content                                                                                                                                                                                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interviewer prompt             | A SaaS company wants GenAI features across customers with strict tenant isolation and budget controls.                                                                                                                                                               |
| Candidate clarifying questions | <ul style="list-style-type: none"> <li>- How are tenants isolated today?</li> <li>- What budgets are contractual?</li> <li>- Can tenants choose model tiers?</li> <li>- What data retention and audit policies apply?</li> <li>- How is billing reported?</li> </ul> |
| Weak answer                    | Add a global token limit and shared cache.                                                                                                                                                                                                                           |
| Strong answer                  | Implement tenant-aware gateway, per-tenant budgets, permission-aware cache, tenant-specific observability, model-routing policy, quota alerts, and isolation tests.                                                                                                  |
| Architecture reasoning         | Tenant gateway -> auth -> budget check -> cache/model route -> tenant-scoped data plane -> cost attribution -> dashboards.                                                                                                                                           |
| Cost/latency trade-offs        | Caching and routing are powerful but dangerous without tenant keys and budget ownership.                                                                                                                                                                             |
| Metrics to monitor             | Cost/tenant, quota usage, cross-tenant access tests, cache hit rate by tenant, P95 latency.                                                                                                                                                                          |
| Final recommendation           | Make tenant_id and permission_signature first-class across every service.                                                                                                                                                                                            |

## Optimize a document summarization pipeline for batch processing

| Case component                 | Content                                                                                                                                                                                                                                             |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interviewer prompt             | Company processes thousands of PDFs nightly and cost is too high.                                                                                                                                                                                   |
| Candidate clarifying questions | <ul style="list-style-type: none"> <li>- Are documents new or changed?</li> <li>- Is full summarization required every time?</li> <li>- What is SLA?</li> <li>- Can work be batched off-peak?</li> <li>- What quality checks are needed?</li> </ul> |
| Weak answer                    | Run all PDFs through the premium model every night.                                                                                                                                                                                                 |
| Strong answer                  | Use content hashing, incremental processing, document dedupe, cheaper summarization model, batch inference, queue budgets, and sampled evaluation.                                                                                                  |
| Architecture reasoning         | Ingest -> hash/dedupe -> parse -> changed-only queue -> batch summarization -> quality sample -> publish.                                                                                                                                           |
| Cost/latency trade-offs        | Batch work optimizes throughput, not interactive latency. Quality can be checked with sampling and strong-model audits.                                                                                                                             |
| Metrics to monitor             | Documents processed, changed ratio, cost/document, queue depth, failure rate, eval score.                                                                                                                                                           |
| Final recommendation           | Never reprocess unchanged documents without a reason.                                                                                                                                                                                               |

## Build a real-time sales assistant with sub-3-second response time

| Case component                 | Content                                                                                                                                                                                                                                                         |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interviewer prompt             | Sales team wants real-time answer suggestions during calls with a sub-3-second target.                                                                                                                                                                          |
| Candidate clarifying questions | <ul style="list-style-type: none"> <li>- What tasks must complete within 3 seconds?</li> <li>- Can data be precomputed before calls?</li> <li>- Is streaming acceptable?</li> <li>- Which facts must be current?</li> <li>- What happens on timeout?</li> </ul> |
| Weak answer                    | Use a large model with live CRM lookup during each request.                                                                                                                                                                                                     |
| Strong answer                  | Precompute account context, keep prompt small, use mid/small model for suggestions, stream quick bullets, run enrichment asynchronously, fallback gracefully.                                                                                                   |
| Architecture reasoning         | Before call: precompute context. During call: small prompt -> fast model -> streamed suggestion. After call: async summary and CRM update.                                                                                                                      |
| Cost/latency trade-offs        | Sub-3-second target requires workflow design, not only model choice.                                                                                                                                                                                            |
| Metrics to monitor             | First-token latency, total latency, cache/precompute hit rate, user accept rate, timeout rate.                                                                                                                                                                  |
| Final recommendation           | Separate real-time suggestions from slower enrichment.                                                                                                                                                                                                          |

## Handle a production incident where LLM costs exploded after launch

| Case component                 | Content                                                                                                                      |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Interviewer prompt             | A new GenAI feature launched yesterday and spending exceeded weekly budget in one day.                                       |
| Candidate clarifying questions | <ul style="list-style-type: none"> <li>- What changed at launch?</li> <li>- Which tenant/feature dominates spend?</li> </ul> |

|                         |                                                                                                                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         | <ul style="list-style-type: none"> <li>- Are retries or agents involved?</li> <li>- Are eval or batch jobs running?</li> <li>- Is there abuse or integration loop?</li> </ul> |
| Weak answer             | Disable the entire AI feature without diagnosis.                                                                                                                              |
| Strong answer           | Activate spend circuit breaker, identify top spend paths, cap output, pause batch/eval jobs, reduce premium routing, apply tenant quotas, then run root-cause analysis.       |
| Architecture reasoning  | Incident dashboard -> cost attribution -> mitigation controls -> customer communication -> permanent guardrails.                                                              |
| Cost/latency trade-offs | Business continuity vs spend control: keep low-cost safe paths online while disabling runaway paths.                                                                          |
| Metrics to monitor      | Cost/minute, token/request, model distribution, retry count, agent steps, tenant usage.                                                                                       |
| Final recommendation    | Budget guardrails must exist before launch.                                                                                                                                   |

## 17. Checklists

---

### Cost optimization checklist

- ☐ Break cost down by tenant, feature, model, prompt version, request path, and batch job.
- ☐ Measure input tokens, output tokens, retries, agent steps, tool calls, embeddings, reranking, and logging overhead.
- ☐ Identify low-value high-cost workflows before reducing quality globally.
- ☐ Set max output length and per-request token budgets.
- ☐ Route simple tasks to smaller models and reserve premium models for complex/high-risk tasks.
- ☐ Deduplicate documents and avoid re-embedding unchanged content.
- ☐ Gate reranking and query rewriting by ambiguity or risk.
- ☐ Introduce safe caching with tenant and permission keys.
- ☐ Create spend anomaly alerts before launch.
- ☐ Review evaluation and batch jobs as part of spend analysis.

### Latency optimization checklist

- ☐ Measure first-token, total latency, retrieval, reranker, model, tool, DB, auth, and post-processing time.
- ☐ Inspect P95 and P99, not only average latency.
- ☐ Create trace waterfall for slow requests.
- ☐ Reduce prompt size and output length where quality allows.
- ☐ Parallelize independent read-only calls.
- ☐ Move long-running work to async jobs.
- ☐ Set timeout budgets per dependency.
- ☐ Warm cold paths for critical demos and production workflows.
- ☐ Use regional deployment and connection pooling.
- ☐ Monitor provider latency separately from application latency.

### RAG optimization checklist

- ☐ Evaluate chunk size and overlap on real documents and real queries.
- ☐ Apply tenant, ACL, document type, and date filters before vector search where possible.
- ☐ Tune top-k using retrieval evals and answer faithfulness.
- ☐ Remove duplicate documents and boilerplate chunks.
- ☐ Use hybrid search for acronyms, IDs, and exact terms.
- ☐ Gate reranking for ambiguous/high-value queries.
- ☐ Compress context while preserving source IDs.
- ☐ Use incremental indexing and content-hash embedding cache.
- ☐ Measure vector DB P95 latency and filter selectivity.
- ☐ Track citation correctness and retrieval recall.

### Agent optimization checklist

- ☐ Set max steps, max tool calls, max cost, and global timeout.
- ☐ Use tool allowlists based on intent.
- ☐ Cache retrieval and tool results within request/session.
- ☐ Use deterministic workflows for common tasks.
- ☐ Use smaller planning model where reliable.
- ☐ Summarize intermediate state rather than passing full trace repeatedly.
- ☐ Parallelize safe read-only tool calls.
- ☐ Add human approval for expensive or sensitive actions.
- ☐ Track agent step count and repeated tool patterns.

- [ ] Write tests for loop and timeout scenarios.

### **Prompt optimization checklist**

- [ ] Remove repeated instructions and unused examples.
- [ ] Separate static and dynamic prompt sections.
- [ ] Set output length defaults and expansion-on-demand behavior.
- [ ] Use structured output only as complex as necessary.
- [ ] Compress retrieved context and summarize chat history.
- [ ] Track prompt template version in every trace.
- [ ] Run prompt token budget tests in CI.
- [ ] Compare quality, retries, parsing errors, and token usage before rollout.
- [ ] Avoid hidden prompt bloat from conversation memory.
- [ ] Keep rollback path for prompt releases.

### **Caching checklist**

- [ ] Define cache type: prompt, semantic, response, embedding, retrieval, tool, session, tenant.
- [ ] Include tenant and permission boundaries in cache keys.
- [ ] Include document, prompt, model, and tool-state versions where relevant.
- [ ] Set TTLs based on freshness and risk.
- [ ] Never cache secrets, tokens, or broad sensitive user-specific answers.
- [ ] Measure hit rate, miss reason, stale response incidents, and invalidations.
- [ ] Prewarm safe high-frequency cache entries.
- [ ] Audit cache isolation for cross-tenant leakage.
- [ ] Document when caching is forbidden.
- [ ] Review caching with security and compliance teams.

### **Observability checklist**

- [ ] Log cost and latency per request path without storing unnecessary sensitive payloads.
- [ ] Track token usage by prompt version and model.
- [ ] Break down latency by auth, retrieval, rerank, prompt build, model, tool, and post-processing.
- [ ] Track cache hit rate and invalidation events.
- [ ] Track retries, timeouts, provider errors, and fallback usage.
- [ ] Track agent step count, tool count, and repeated tool calls.
- [ ] Build executive, engineering, tenant, incident, and model dashboards.
- [ ] Add alerts for spend anomalies and P95/P99 latency regressions.
- [ ] Retain traces according to privacy policy.
- [ ] Use sampling where full tracing is too expensive.

### **Customer communication checklist**

- [ ] Acknowledge business impact before explaining technical details.
- [ ] Separate quality, cost, latency, security, and workflow-fit issues.
- [ ] Explain trade-offs using options, not excuses.
- [ ] Avoid blaming the model provider without evidence.
- [ ] Show measured bottlenecks and before/after targets.
- [ ] Connect optimization to ROI, adoption, and budget predictability.
- [ ] Document immediate mitigation and long-term prevention.
- [ ] Use plain language for executives and detailed traces for engineers.
- [ ] Set expectations for high-risk workflows that require slower verification.
- [ ] Be transparent about caching, data retention, and tenant isolation.

**Pre-production readiness checklist**

- ☐ Load test realistic traffic, documents, tenants, and prompt sizes.
- ☐ Define latency and cost SLOs by workflow.
- ☐ Set budgets, quotas, rate limits, token limits, and agent limits.
- ☐ Validate permission-aware retrieval and caching isolation.
- ☐ Run prompt/model regression evaluations.
- ☐ Review observability dashboards and alerts.
- ☐ Test provider fallback and graceful degradation.
- ☐ Test batch job budgets and retry behavior.
- ☐ Prepare incident runbooks and customer communication templates.
- ☐ Run security/compliance review for logging, caching, and retention.

**Executive demo readiness checklist**

- ☐ Use a stable dataset that matches the business scenario.
- ☐ Warm critical services and caches.
- ☐ Rehearse with production-like network conditions.
- ☐ Measure latency for every demo path.
- ☐ Prepare fallback path for provider or network delays.
- ☐ Use concise answer defaults and streaming where helpful.
- ☐ Avoid uncontrolled live agent workflows unless tested.
- ☐ Have a clear ROI and trade-off explanation ready.
- ☐ Do not overclaim production readiness from a demo.
- ☐ Capture feedback and convert it into measurable next steps.

## 18. Final Senior-Level Mental Model

A senior GenAI FDE does not optimize blindly. They measure first, decompose the system, identify the largest cost and latency drivers, and make trade-offs visible to customers. The strongest candidates can explain why a faster model is not always the answer, why more context is not always better, why agents need budgets, and why caching must be secure before it is useful.

### The senior mental model

- **Measure before changing:** Use traces, cost attribution, token metrics, and P95/P99 latency before proposing fixes.
- **Break latency into components:** Auth, retrieval, reranking, prompt construction, first-token, generation, tool calls, post-processing, and network return.
- **Break cost into drivers:** Input tokens, output tokens, embeddings, reranking, retries, agent steps, tools, logging, evals, batch jobs, and infrastructure.
- **Optimize workflow before model:** Many wins come from better routing, async design, caching, and reducing unnecessary steps.
- **Use smaller models where possible:** Match model strength to task complexity, risk, and quality requirement.
- **Cache carefully:** Caching without tenant and permission safety can become a security incident.
- **Control agent behavior:** Agents need step limits, tool budgets, traces, deterministic routes, and stopping rules.
- **Set budgets and guardrails:** Spend should be predictable by tenant, feature, user, and request path.
- **Communicate trade-offs clearly:** Customers need options with benefits, risks, and business impact.
- **Connect optimization to business value:** The purpose is adoption, ROI, reliability, security, and sustainable scale.

#### Final interview-ready closing answer

"My default approach is to avoid blind optimization. I would first instrument the workflow, break latency and cost into measurable components, identify the largest drivers, and then choose the least risky optimization. I would usually start with workflow design, prompt size, retrieval tuning, model routing, caching, and guardrails before moving to infrastructure scaling or model changes. The goal is not simply cheaper and faster AI; the goal is a system that is fast enough, accurate enough, secure enough, and economically sustainable for the customer."

## One-page field summary

| Question                        | Senior answer                                                                                    |
|---------------------------------|--------------------------------------------------------------------------------------------------|
| Where is the latency?           | Trace the full path: auth, retrieval, rerank, model, tools, post-processing, network.            |
| Where is the cost?              | Attribute spend by tokens, model, retries, agent steps, embeddings, tools, evals, infra, tenant. |
| What should be optimized first? | Largest measured bottleneck that can be improved without breaking quality/security.              |
| Should we use a smaller model?  | Yes for simple low-risk tasks if evals confirm quality; escalate for complex/high-risk tasks.    |
| Should we cache?                | Yes only with tenant, permission, freshness, and retention controls.                             |
| Should we use agents?           | Only when deterministic workflows are insufficient; agents must be budgeted and traced.          |
| How do we convince executives?  | Show cost predictability, latency targets, adoption impact, and ROI metrics.                     |

## Final practice drill for candidates

Before every GenAI FDE interview, practice explaining one optimization incident in this structure: business impact, measured bottleneck, technical root cause, immediate mitigation, long-term prevention, and executive/customer communication.

| 60-second answer structure | Example wording                                                                                                                        |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 1. Frame the problem       | The system was useful, but cost and latency were not yet production-ready. I treated this as a measured workflow optimization problem. |
| 2. Decompose the path      | I separated retrieval, reranking, model inference, tool calls, retries, caching, and post-processing.                                  |
| 3. Identify the driver     | The largest driver was repeated retrieval and agent tool loops, not the base model alone.                                              |
| 4. Fix safely              | I added step limits, cached retrieval within the request, routed simple tasks to a smaller model, and capped output length.            |
| 5. Prove improvement       | I compared P95 latency, cost/request, quality evals, cache hit rate, and user success rate before and after.                           |
| 6. Prevent recurrence      | I added budgets, alerting, prompt/token regression checks, and an incident dashboard by tenant and feature.                            |

### Closing principle

Premium GenAI engineering is not about using the most powerful model everywhere. It is about building a measured, secure, cost-aware, latency-aware system that customers can trust, afford, adopt, and scale.